Plan práctico de pruebas automáticas con CI/CD

Caso seleccionado: CitaSalud | Sistema web de citas médicas

1. Descripción de la aplicación

CitaSalud permite a los pacientes ingresar al sistema, revisar los horarios disponibles de cada médico y reservar citas. Al confirmar una reserva, la aplicación la registra y evita que un mismo horario se asigne a dos pacientes.

2. Plan de pruebas automáticas

Se automatizarán tres funcionalidades. Cada prueba se ejecutará cuando el equipo envíe cambios al repositorio y permitirá comprobar tanto casos válidos como situaciones que deben rechazarse.

Función y tipo	Preparación y datos de prueba	Acciones automáticas	Resultado esperado
P01 · Inicio de sesión Unitaria	Simular un usuario registrado con contraseña válida. Preparar también una contraseña incorrecta. No se usa la base de datos real.	Ejecutar la validación de credenciales con ambos conjuntos de datos y comprobar la respuesta del componente.	Acepta las credenciales correctas y rechaza las incorrectas sin autorizar el acceso.
P02 · Consultar horarios Integración	Crear una base de datos de prueba con un médico y dos horarios: 09:00 ocupado y 10:00 libre.	Consultar la disponibilidad a través del servicio de la aplicación y verificar los registros devueltos.	Muestra 10:00 como disponible y excluye 09:00 porque ya existe una cita.
P03 · Agendar una cita Interfaz (E2E)	Usar un paciente de prueba, un médico y el horario libre de las 10:00. Iniciar la aplicación de prueba.	Con un navegador automático, iniciar sesión, seleccionar fecha y hora, confirmar y luego intentar reservar el mismo horario otra vez.	La primera reserva se confirma y aparece en la lista. El segundo intento no permite duplicarla.

3. Condiciones para ejecutar el plan

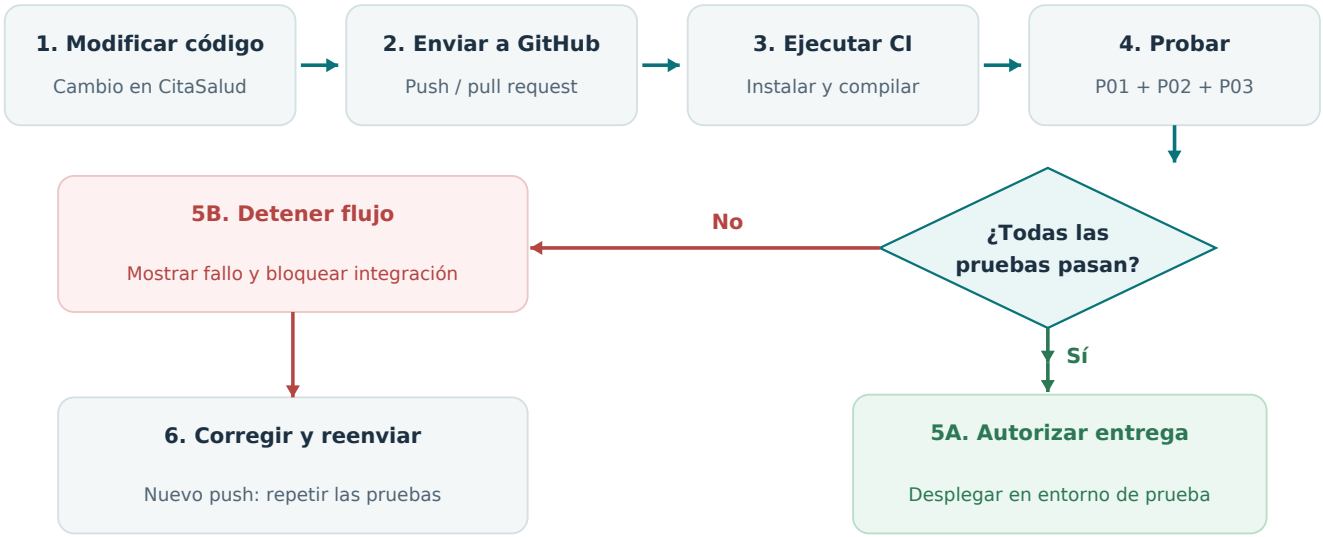
Entorno: repositorio GitHub, ejecución automática con GitHub Actions, datos ficticios en una base de pruebas y navegador automatizado con Playwright para P03. Los datos de prueba se restablecen antes de cada ejecución.

Criterio de aprobación: P01, P02 y P03 deben terminar correctamente. Si falla una, el cambio no se integra ni pasa a despliegue; se conserva el reporte para corregirlo.

Flujo de ejecución CI/CD

Cómo se aplica el plan desde un cambio de código hasta la entrega de una versión validada.

4. Diagrama del proceso



5. Descripción del flujo

1. Cambio y envío. El programador modifica una función, guarda el cambio y lo envía a GitHub mediante push o una solicitud de integración.

2. Activación del pipeline. GitHub Actions detecta el envío, descarga el proyecto, instala dependencias y comprueba que compile.

3. Pruebas en orden. Ejecuta P01 (inicio de sesión), P02 (horarios con base de pruebas) y P03 (reserva desde el navegador). Se guardan sus resultados.

4. Decisión automática. Si alguna prueba falla, se detiene el proceso, se muestra el error y no se permite integrar el cambio.

5. Continuación o corrección. Si todas pasan, se autoriza la entrega al entorno de pruebas. Si alguna falla, el desarrollador corrige el código y un nuevo envío reinicia el ciclo.

6. Reflexión final

¿Qué beneficios aporta tener CI/CD en este caso?

Cada cambio en CitaSalud se verifica de la misma manera y se detectan errores antes de entregar una versión. Esto facilita mantener estables el acceso, la disponibilidad y las reservas.

¿Qué problemas se evitarían al automatizar las pruebas?

Se reduce el riesgo de aceptar credenciales incorrectas, mostrar horarios ocupados como libres o generar citas duplicadas después de modificar el sistema.

¿Qué automatizaría primero y por qué?

Empezaría por P02, la consulta de horarios, porque la disponibilidad correcta es indispensable para evitar reservas superpuestas y puede comprobarse rápidamente con datos de prueba.